PART 1 · CHAPTER 3

Setting Up Your Pro Environment

CLAUDE.md, persistent memory, and the project context system that means you never repeat yourself again

Every developer who starts using Claude Code goes through a predictable frustration. The first session is great. Claude Code picks up on your project quickly, follows your preferences, and produces good results. The second session — different story.

You open a new terminal. You start Claude Code. It has no memory of your previous conversation. It does not know that you use async/await everywhere, that your error handler lives at src/middleware/error.ts, that you never add console.log statements to production code. You have to explain all of this again, from scratch.

This is the most common reason developers give up on Claude Code after a promising start. It feels like training the same employee from zero every morning.

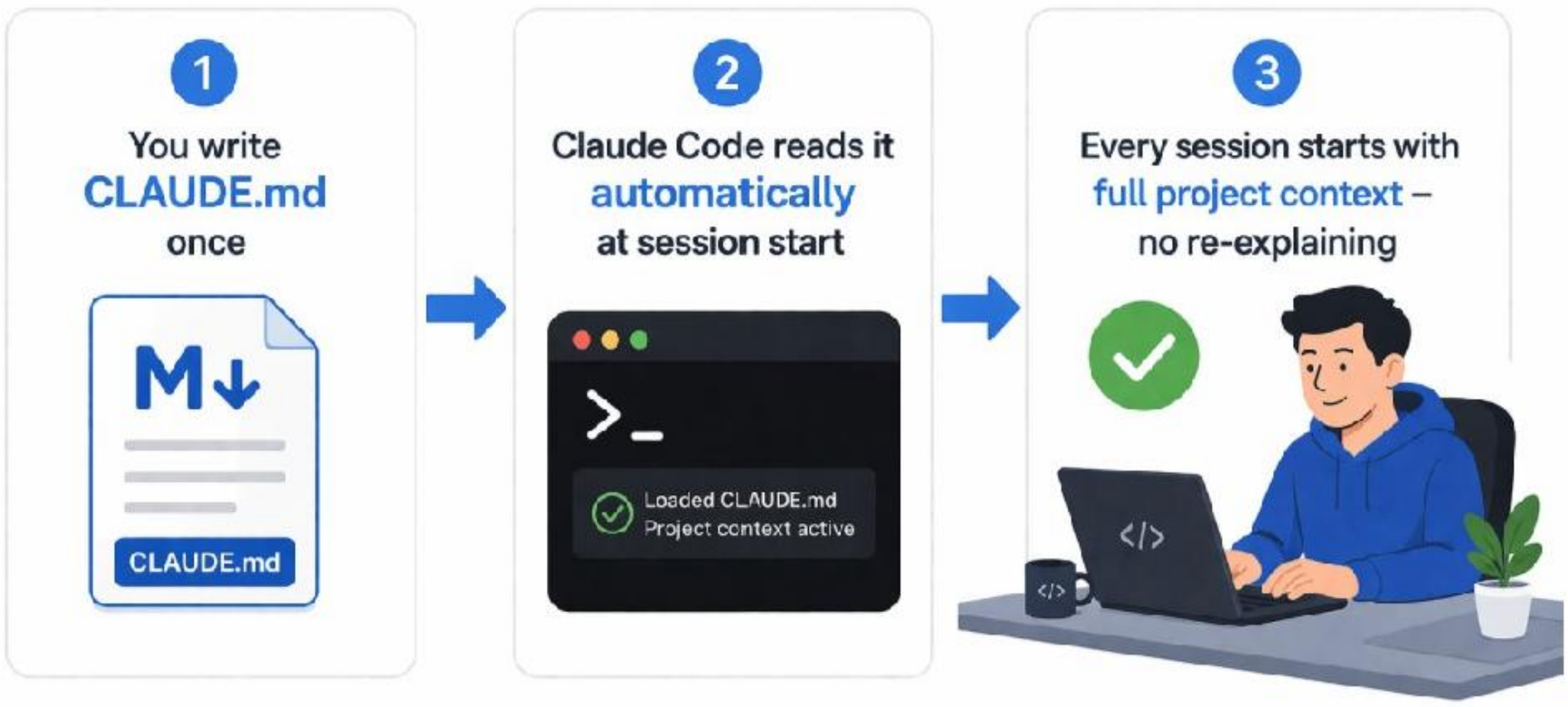
The solution is CLAUDE.md — and understanding how to use it properly is the difference between a tool you use sometimes and a tool that becomes the foundation of how you work.

What is CLAUDE.md?

CLAUDE.md is a plain Markdown file that lives at the root of your project directory. Every time you start a Claude Code session in that directory, it reads this file automatically, before anything else happens.

Think of it as the onboarding document for your project — the document you would give a new developer on their first day to bring them up to speed on everything they need to know to work effectively on this codebase. Except Claude Code reads it in full at the start of every single session, with perfect retention.

Whatever you put in CLAUDE.md, Claude Code will know. Your tech stack, your conventions, your patterns, your constraints, your file structure, your testing approach, your deployment setup. Once it is in CLAUDE.md, you never have to explain it again.



Anatomy of a great CLAUDE.md

A great CLAUDE.md covers six areas. Let's look at each one, then build a complete example.

1. Project overview

One to three sentences describing what the project is, who uses it, and what it does. This gives Claude Code the semantic context to make better decisions — it knows whether this is a consumer app, an internal tool, a public API, or a developer library.

2. Tech stack

Every library, framework, and runtime that is relevant to how code is written. Be specific about versions if they affect behavior. Include both what you use and what you have explicitly chosen not to use — this prevents Claude Code from adding an alternative library when it is solving a problem where you have already made a tool choice.

3. Architecture and file structure

Describe the folder structure and the pattern you follow. If you use MVC, say so. If you use feature-based folders rather than type-based folders, say so. If there is a specific file that is the entry point for specific types of code (like a main router, or a central service registry), identify it.

4. Code conventions

How is code written on this project? Async/await or promises? Classes or functions? Named exports or default exports? What is the error handling pattern? What is the logging pattern? What naming conventions do you use? These details determine whether Claude Code-generated code fits seamlessly into your codebase or stands out as foreign.

5. Testing approach

What testing framework do you use? Where do tests live? What gets tested and what does not? What mocking library do you use? Do tests need to run before commits? This section prevents Claude Code from writing tests in a style that conflicts with your existing test suite.

6. Standing constraints

The things that are always true. Never do X. Always do Y. If it touches Z, check with me first. These are the project-level negative constraints that apply to every task, not just specific ones.

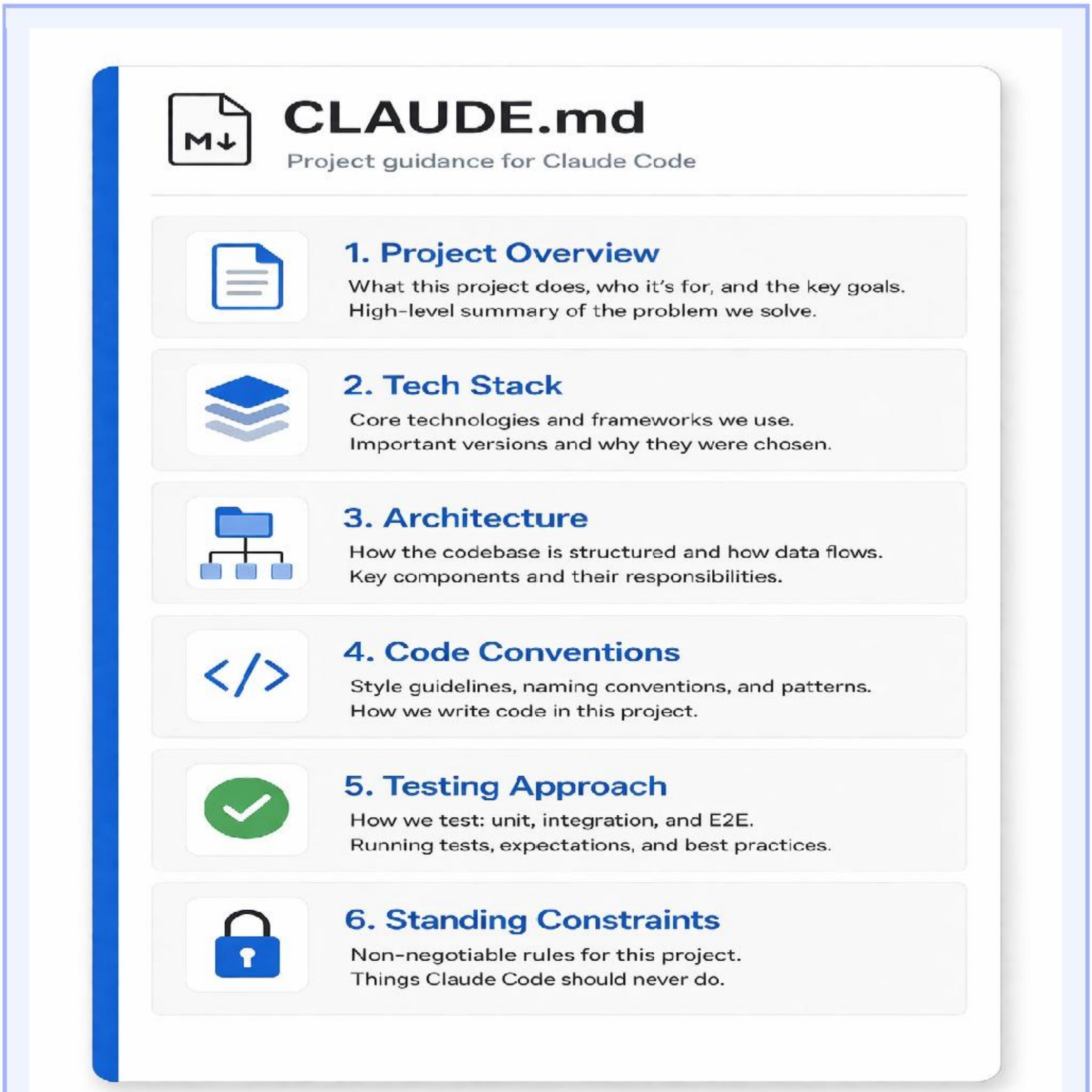
A complete CLAUDE.md example: the Acme API project

Here is a full CLAUDE.md for a real-world-style Node.js API project. Use this as your template and adapt it to your project.

```
# Acme API — Claude Code Project Context

## Project overview
Acme API is a B2B REST API serving the Acme web dashboard
and mobile app. It handles user authentication, subscription
management, and data analytics for ~2,000 business accounts.

## Tech stack
- Runtime: Node.js 20 (LTS)
- Language: TypeScript (strict mode)
- Framework: Express 4
- Database: PostgreSQL 16 via Prisma ORM
- Cache: Redis 7 via ioredis
- Auth: JWT (jsonwebtoken) — NO sessions
- Validation: Zod
- Testing: Jest + supertest
- Logging: pino (never console.log in production code)

## Architecture
Feature-based folder structure under src/:
  src/features/{feature}/
    controller.ts   — request/response handling only
    service.ts      — business logic
    repository.ts   — database queries only
```

```
    routes.ts        — Express router
    types.ts         — TypeScript types for this feature
    *.test.ts        — integration tests

Central error handler: src/middleware/error.ts
Auth middleware: src/middleware/auth.ts
Prisma client: src/lib/prisma.ts (import from here)
Redis client: src/lib/redis.ts (import from here)

## Code conventions
- Always async/await — never .then()/.catch() chains
- Always named exports — never default exports
- All errors thrown as AppError from src/lib/errors.ts
- Request validation with Zod before any business logic
- Repository functions return plain objects, not Prisma types
- No any types — use unknown if type is genuinely unknown

## Testing
- Tests are integration tests using supertest against real DB
- Test database is separate (see .env.test)
- Each test file creates and cleans up its own test data
- No unit tests for repositories (they are DB integration by nature)
- Run: npm test

## Standing constraints
- Never install new packages without flagging it to me first
- Never modify src/database/migrations/ — I handle migrations
- Never add console.log — always use logger from src/lib/logger.ts
- Never modify .env files
- All new routes must go through the central router in src/app.ts
- Prisma schema changes require a migration — flag them to me
```

Notice what this file does: it answers every question Claude Code would otherwise have to guess at. When it writes a new feature, it knows exactly where to put it, what pattern to follow, which error class to use, which libraries to import from, and what the tests should look like. The result is code that looks like it was written by a developer who has been on the project for years.



Four starter CLAUDE.md templates

Below are ready-to-use CLAUDE.md templates for four common project types. Copy the one that matches your project and fill in the bracketed fields. All four templates are also available in the downloadable resources.

Template 1: Python FastAPI project

```
# [Project Name] — Claude Code Context

## Overview
[What the project does and who uses it]
```

```
## Tech stack
- Python 3.11+, FastAPI, SQLAlchemy 2 (async), PostgreSQL
- Pydantic v2 for validation, alembic for migrations
- pytest + httpx for testing, ruff for linting

## Architecture
app/routers/{feature}.py  — route handlers
app/services/{feature}.py — business logic
app/models/{feature}.py   — SQLAlchemy models
app/schemas/{feature}.py  — Pydantic schemas

## Conventions
- Async everywhere (async def, await, AsyncSession)
- Dependency injection via FastAPI Depends()
- All config from environment via app/core/config.py
- Errors raised as HTTPException with detail dict

## Constraints
- Never use sync SQLAlchemy sessions
- Never import directly from database — use repositories
- Always run ruff check before considering a task complete
```

Template 2: React + TypeScript frontend

```
# [Project Name] — Claude Code Context

## Overview
[What the frontend does]

## Tech stack
- React 18, TypeScript (strict), Vite
- TanStack Query for data fetching, Zustand for global state
- Tailwind CSS, shadcn/ui components
- Vitest + React Testing Library

## Architecture
src/features/{feature}/
  components/  — UI components for this feature
  hooks/       — custom hooks
  api.ts       — API call functions
  types.ts     — TypeScript types

## Conventions
- Functional components only — no class components
- Custom hooks for all data fetching (use TanStack Query)
- All API calls centralized in feature/api.ts files
- No inline styles — Tailwind classes only
- Prefer composition over prop drilling

## Constraints
- Never use useEffect for data fetching — use TanStack Query
- Never add new UI component libraries
- All forms use react-hook-form + Zod validation
```

Template 3: Full-stack Next.js project

```
# [Project Name] — Claude Code Context

## Overview
[What the application does]

## Tech stack
- Next.js 15 (App Router), TypeScript, Tailwind CSS
- Prisma + PostgreSQL, NextAuth.js for auth
- Shadcn/ui, React Server Components where possible
- Playwright for E2E, Vitest for unit tests

## Architecture
app/(routes)/   — page routes and layouts
app/api/        — API route handlers
components/     — shared React components
lib/            — utilities, DB client, auth config
prisma/         — schema and migrations

## Conventions
- Prefer Server Components; use 'use client' sparingly
- Data mutations via Server Actions, not API routes
- Database access only in server-side code (never client)
- All shared types in lib/types.ts

## Constraints
- Never expose Prisma types to the client
- Never use pages/ directory — App Router only
- All environment variables go in .env.local (not committed)
```

Template 4: Freelance client project

```
# [Client Name] — [Project Name] Claude Code Context

## Overview
[What the project does. Who the client is. Delivery date: X]

## Tech stack
[Stack agreed with client]

## Client standards
- Code must be clean enough for client's team to maintain
- Comments required on any non-obvious logic
- No external services without client approval
- Design: [reference design file or style guide]

## Deliverables
- [List what needs to be delivered]

## Constraints
- No paid third-party services — client controls integrations
```

```
- All API keys go in environment variables, never in code
- No experimental or alpha-version packages
```

Memory files: persisting knowledge across sessions

Beyond CLAUDE.md, Claude Code supports memory files — text files that store specific knowledge Claude Code has accumulated or that you have explicitly given it. Unlike CLAUDE.md, which you write and maintain manually, memory files can be updated by Claude Code itself during a session.

Memory files are useful for storing things that change over time: decisions you have made and the reasoning behind them, known issues and their workarounds, API endpoint documentation, environment-specific configuration notes, and anything else that needs to persist between sessions but is too dynamic or too detailed for CLAUDE.md.

Creating a memory file

You can create a memory file at any point in a session by instructing Claude Code:

```
> Save this to a memory file: we decided to use Redis pub/sub
  for real-time notifications instead of WebSockets because the
  client's hosting environment does not support persistent
  connections. File: .claude/decisions.md
```

Claude Code will create the file and reference it in future sessions when relevant. You can also ask Claude Code to read a memory file explicitly at the start of a session:

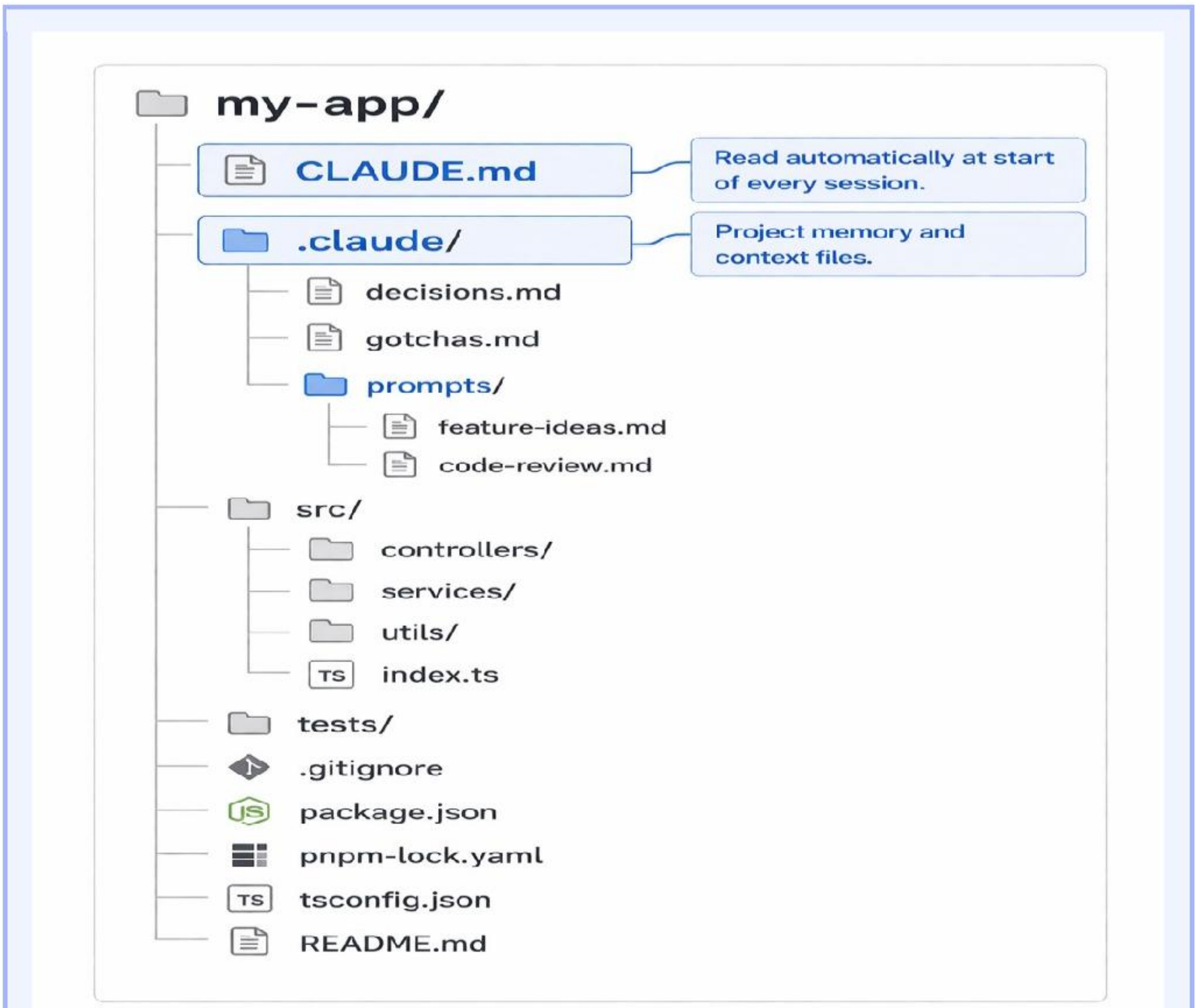
```
> Read .claude/decisions.md before we start.
```

The .claude/ directory

A well-organized Claude Code project has a .claude/ directory at the project root that contains all Claude-specific files:

```
.claude/
  decisions.md      — architectural decisions and reasoning
  gotchas.md        — known issues, workarounds, traps
  api-reference.md — internal API documentation
  progress.md       — current state of work in progress
  prompts/          — reusable prompt templates for this project
```

Add .claude/ to your .gitignore if you do not want to share these files with your team, or commit them if you do — they are genuinely useful context for any developer working on the project, human or AI.



Verifying your setup works

Once you have created your CLAUDE.md file, take five minutes to verify it is working correctly before relying on it. Start a fresh Claude Code session and ask:

```
> Without me telling you anything, describe this project:
  the tech stack, the folder structure, the conventions,
  and the things I've asked you never to do.
```

Claude Code should accurately describe everything you have put in CLAUDE.md. If anything is missing or wrong, update the file and test again. This takes five minutes and prevents hours of frustration from sessions where Claude Code quietly ignored or misunderstood your context.

PRO SETUP TIP

Create your CLAUDE.md at the very start of a project, before writing any code. A CLAUDE.md built at the beginning shapes every subsequent session. A CLAUDE.md written after months of

development has to retroactively describe decisions that were never articulated — it is harder to write and less effective.

Chapter 3 practice: build your CLAUDE.md

Before moving to Part 2, take thirty minutes to create a CLAUDE.md for a project you are actively working on. If you do not have one, use the freelance client template above and fill it in for a hypothetical project.

Cover all six areas: project overview, tech stack, architecture, code conventions, testing approach, and standing constraints. Then start a fresh Claude Code session and run the verification prompt above.

When Claude Code accurately describes your project back to you without any prompting, your pro environment is set up. From this point on, every session you start on this project will begin with that full context already in place.

The investment you make in the next thirty minutes will pay dividends in every Claude Code session you run on this project for as long as the project exists.

CHAPTER 3 SUMMARY

CLAUDE.md is a Markdown file at your project root that Claude Code reads automatically at the start of every session. It is the onboarding document for your project — covering tech stack, architecture, conventions, testing approach, and standing constraints.

Memory files in a .claude/ directory let you persist knowledge that changes over time. A well-configured environment means Claude Code knows your project before you type your first prompt.

With your environment set up, you are ready for Part 2 — where we put all of this to work on real code: generating, debugging, testing, and refactoring at professional speed.